Natural Computing 2

Ivar van der Spoel & Ivor Brouwer

Group number 21

December 2023

Abstract

This scientific report presents a study on optimization algorithms, focusing on a comparison between Simulated Annealing and Random Search. The research investigates the performance of different parameter configurations of Simulated Annealing and assesses their effectiveness in solving instances of the Maximum Cut problem, a submodular optimization problem. The algorithms are evaluated on five distinct problem instances, and the results indicate the superiority of Simulated Annealing, particularly with a specific parameter configuration. The study also explores the challenges posed by the vast search space in Random Search and provides insights into the convergence behavior of the algorithms. Overall, the findings contribute to a better understanding of the optimization landscape and offer valuable considerations for algorithm selection and parameter tuning.

1 Introduction

This is the scientific report for Assignment 2 of Natural Computing. In this assignment, we were required to compare optimization algorithms. To accomplish this, we had to develop our own implementation of an optimization algorithm and a random search algorithm for comparison. The evaluation involved benchmarking three different parameter configurations of our algorithm and testing them on five distinct instances of the Maximum Cut problem. Additionally, we include a literature study on the real-world applications of our algorithm.

Background

Below, you'll find explanations of concepts that will be frequently used in this paper.

The focus of this report is on an **optimization problem**. In such a problem, the task is to find the best solution among a set of solutions, where "best" is typically defined based on an objective function. These problems arise in numerous real-world scenarios where resources are limited, and the goal is to maximize or minimize a certain quantity [3]. Submodular function problems represent a class of these optimization problems.

A **submodular problem** involves optimizing a function that describes the relationship between sets of inputs and a real-valued output. Specifically, a submodular function has diminishing returns, meaning that the relative increase in value when adding elements to a set is larger when the set is smaller, and this increase diminishes as the set grows. Adding more elements may even lead to a decrease in the total value.

To tackle an optimization problem, we can use **simulated annealing**. MathWorks describes simulated annealing as "A method for solving unconstrained and bound-constrained optimization problems. The method models the physical process of heating a material and then slowly lowering the temperature to decrease defects, thus minimizing the system energy" [7]. The algorithm initiates with an initial solution and iteratively explores neighboring solutions, accepting both better and worse solutions with a certain probability. The probability of accepting worse solutions decreases over time, simulating the annealing process. This enables the algorithm to escape local optima and explore the solution space more effectively.

2 Problem Description

The maximum cut problem is a specific instance of a submodular problem. The maximum cut problem is described as a graph with two sub-graphs, such that the number of edges between these two sub-graphs (i.e. the "cut") is maximized. Moving nodes to a different set could both increase and decrease the fitness, so this problem is a submodular problem.

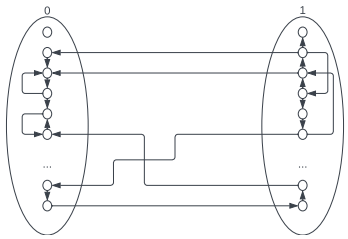


Figure 1: Example Max Cut

In figure 1 an example graph is drawn, where two subsets *0* and *1* of nodes from the complete graph are given, and the count of edges between one another is the count of lines between the two sets.

Our task is to develop two distinct algorithms: one that randomly selects sets *0* and *1*, and another that employs an algorithm to strategically choose these sets, aiming to maximize the number of edges between them.

3 Literature Study

The application of simulated annealing in the context of police command districting is a notable endeavor by D'Amico et al. (2002) [5]. This endeavour is very important, because it helps improving efficiency of emergency services.

3.1 Simulated Annealing in practice

The approach of D'Amico involves a simulated annealing search methodology for the redesign of police command boundaries. At each iteration of the simulated annealing heuristic, the authors employ a variant of the Patrol Car Allocation Model (PCAM) to strategically allocate patrol resources to the current district partition. This allocation aids in the evaluation of the expected performance of the district design. The study utilizes a case study based on data from the Buffalo Police Department (BPD) to establish the effectiveness of simulated annealing.

3.2 Algorithm selection

The researchers opted for simulated annealing, because this technique was newly developed and showed promise. The selection of simulated annealing for police district design stems from practical considerations and limitations associated with existing tools, namely PCAM (Patrol Car Allocation Model) and DH (Desktop Hypercube). Recognizing the need to overcome the combinatorial challenges inherent in district design, the researchers turned to simulated annealing. Unlike the "human-factor" of DH and the resource-centric focus of PCAM, simulated annealing does not have these flaws.

The paper also mentions another algorithm, tabu search [4]. In contrast to the employed simulated annealing approach, tabu search emerges as an alternative in police districting. Simulated annealing, tabu search, and genetic algorithms represent a subset of metaheuristic methods known for their success in navigating the complexities of combinatorial optimization problems.

3.3 Success and results

The simulated annealing methodology proposed for police command districting appears to be successful. The authors observed a drastic improvement, reducing the maximum workload disparity from over 50% to only 14%. Under optimal car allocations, the simulated annealing approach led to the identification of an improved district design. Despite the changes in district configurations, the response time feasibility constraints were maintained, ensuring that there was no increase in the maximum response time under the current BPD operations. The proposed district configurations were not only effective in achieving workload balance but also addressed operational inefficiencies, such as the dispatching quirk related to calls for service on the Buffalo Skyway.

3.4 Critique and improvement

The choice of simulated annealing in this context seems justified given the complexities and constraints associated with the problem. Namely, the dynamic adjustment property of the algorithm is beneficial for finding optimal district configurations. Moreover, simulated annealing is capable of exploring a broad solution space and avoiding local maxima. This is essentially the reason why we are researching this algorithm as well.

However, deciding which algorithm to choose depends on the specific characteristics of the problem, available data, and the goals of the optimization. The authors' efforts to improve their methodology, including suggestions for refining the algorithm using geographical information system (GIS) software, indicate a commitment to optimization and continuous improvement. A recommendation would be to employ and integrate the different tools that the researchers had access to and find a middle-ground. The tools in question are namely PCAM and DH.

4 Methods

We implemented two algorithms: random search and a customized version of Simulated Annealing.

For random search, we designed an algorithm that, for each variable in the search space (n), randomly selects either *0* or *1*. This selection is then assessed for fitness. Implementation-wise, we utilized an array of size n and set each position to a random value of *0* or *1*.

In our adapted version of simulated annealing, inspired by Practical Study 5 from the Natural Computing course [6], we initialize an array of size n with random values. Mutations are applied over a specified number of iterations, altering one value to its opposite. Our algorithm accepts a worse solution if a randomly generated value between *0* and *1* is lower than our temperature, typically starting at *1*. The algorithm includes several parameters, such as temperature, temperature step, iterations,

the number of mutations per iteration, and an accept function. We vary these parameters to assess their impact on the algorithm's fitness.

To analyze our results, we employ the IOHAnalyser toolkit provided by LIACS [1]. This tool facilitates comprehensive data analysis, allowing us to examine our results within a fixed budget, evaluate the final outcomes, and assess empirical attainment.

4.1 Experimental setup

The parameterized versions of our simulated annealing algorithm which will be tested can be found below in table 1. p1, p2 and p3 are parameter configurations designed such that we can assess certain properties. Note that each configuration is set to have 20.000 iterations. Each experiment will run 10 times, because our algorithms are stochastic so we need to perform multiple runs. Namely, p1 is designed to accept "bad" configurations in the first 100 iterations, to try and move away from low peaks. After this, p1 only accepts better mutations. Moreover, p2 is designed to have a fast decrease in temperature initially, and then cool down at a slow rate and the accept function will become less likely, but still possible, to be accepted. In p3 we try to increase the amount of permutations to increase the "jumps" of improvements or deterioration. It also tests the influence of changing the temperature step from a linear to an exponential decrease.

	temperature step	mutations per iteration	accept function
p1	-0.01	1	rand(0, 1) < temperature
p2	-temperature/iterations	1	rand(0, 1) < $\exp\left(\frac{new_value - current_value}{temperature}\right)$
p3	*0.99	2	rand(0, 1) < $\exp\left(\frac{new_value - current_value}{temperature}\right)$

Table 1: Parameterized algorithms that will be tested.

4.2 Expectations

Our expectation for the temperature is that it might settle too quickly for p1 which might cause the algorithm to stay at one optimum quickly, which might not be the global optimum since the temperature will be 0 quite fast. With an increase in mutations per iterations for p3 we expect it to negatively impact the fitness since we expect the algorithm to mutate too fast and because of this it might skip over better solutions. We expect p2 to be the most optimal algorithm because it has a better accept function and temperature step function.

Our expectation for the Random Search algorithm is that it will not perform well, as there are 2^{800} permutations. This is an incredibly big number, and we think that only a small percentage of these solutions are considerably good. So, the chance of finding this one at random is very unlikely.

5 Results

Below follow the results for the 5 different problems from the IOH library we tested using our 3 different parameterized algorithms and random search.

5.1 Simulated Annealing versus Random Search

We will briefly evaluate every problem on their own, and give a final observation on all of the problems at the end. The graphs are all in figure the figures show the mean fitness between 10 runs of every algorithm and +/- standard deviation compared to the amount of function evaluations.

Problem 2000 In figure 3a we can see the graph for problem 2000, p3 slows down earlier than both p1 and p2, but gains slightly on p1 later on. p2 ends up significantly higher than the other algorithms. Random Search is clearly worse than the other algorithms.

Problem 2001 problem 2001 is shown in figure 3b. The graph closely resembles the graph of problem 2000, only p3 is slightly lower than p3 in problem 2000.

Problem 2002 problem 2002 in figure 3c differs from problem 2000 and 2001 at lower function evaluations. Here all algorithm increase in fitness at a more similar rate, but still the end results do resemble the earlier problems.

Problem 2003 Figure 3d for problem 2003 shows a change from problem 2002, here the curve for p3 is moved up slightly and because of this it learns at almost the same rate and p3 even ends up a bit higher than p1.

Problem 2004 The most interesting observation for problem 2004 in figure 3e is that the standard deviation is higher than all the other problems, indicated by the transparent areas around the curves.

All problems Overall we can see that every algorithm performs better than randomly searching the search space. As anticipated, Random Search did not do much better than it's initial fitness. As we expected is p2 outperforming p1 from 5000 evaluations forward or earlier in all problems, from this point there is even no overlap from their standard deviations, so p2 is significantly better than p1 in obtaining a high fitness. This has to do with the improved temperature steps and the improved accept function for worse solutions. It is also interesting to see that p3 starts outperforming p1 slightly from 15000 evaluations forward in most problems and significantly in problem 2000, 2003 and 2004. This might have to do with the bigger changes it does, since it does more mutations per iteration, when its already moving towards an optimum solution.

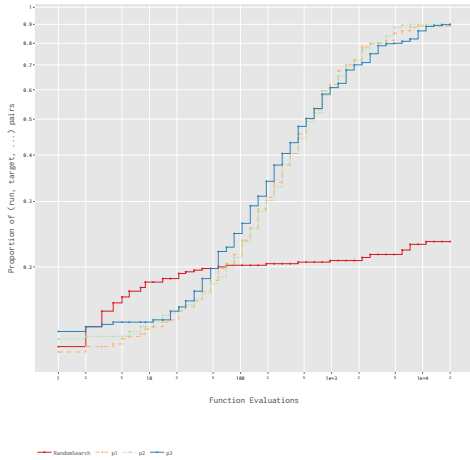
5.2 Worthwhile to improve

In submodular set functions finding a global optima can take time to find. Improving the current fitness could take one iteration to many. Specifically, in simulated annealing it is true that by increasing the amount of iterations we will approach the best result [2]. This is apparent in our algorithm as well, as by increasing the iterations five fold, we approach a better result. However, the gain of reaching this result becomes increasingly lower and it might not be beneficial to increase the amount of evaluations.

5.3 Approaching target based on iterations

Figure 2a indicates the performance of the algorithms with varying time budgets. It shows the relationship between the available budget (function evaluations) and the fraction of quality the runs reach compared to the best target solution.

Table 2b shows the total area under each curve (AUC). The AUC represents the cumulative quality across all budget points and provides a measure for the algorithm effectiveness. Algorithms with a high AUC have a higher quality across different time budgets.



(a) ECDF graph including the different algorithms.

	ID	x	auc
1	RandomSearch	20001.00	4520.82
2	p1	20001.00	16872.32
3	p2	20001.00	17076.00
4	p3	20001.00	16567.03

(b) The approximated Area Under the Curve of the displayed ECDF.

Since p2 has the highest AUC and is closer to the best target value at most points its clear that its the most robust and highest quality algorithm of the four, closely followed by p1.

5.4 Difficult problems

To assess which problems are the most difficult, we have to look at the optimization rates of each algorithm and determine which problem caused the most trouble.

problem	fitness	avg mean
2000	11494	10992.85
2001	11487	11004.85
2002	11492	11002.08
2003	11517	11009.08
2004	11503	10996.38

Table 2: Problems highest fitness and average mean

Albeit marginal, we can see in table 2 that problems 2000, 2001 and 2002 have the lowest fitness. It only comes down to tens of edges, but in these optimization problems a fraction of a percentage can be a big deal. Moreover, problem 2000 has the lowest average mean. Besides, in figure 3 it is apparent that 3a and 3b have the steepest optimization curves for p3. The "gap" is visibly bigger than in the other problems. Having this in mind, we can conclude that overall problem 2000 is the most difficult one, admittedly only by a small margin.

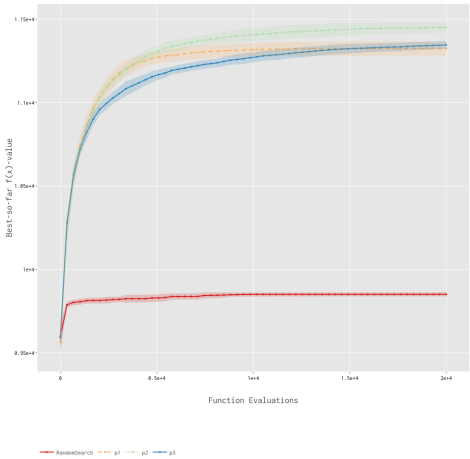
6 Conclusion

In conclusion, the research focused on the performance of different parameterized versions of our optimization algorithm and the random search algorithm. To gain more insight we experimented on 5 different versions of the maximum cut problem. The differences in difficulty among the problems were found to be minimal. However, problem 2000 proved to be most challenging to optimize, shown by the steeper curve and the lowest mean overall. Random Search demonstrated poor performance in comparison to the other algorithms. The vast search space with 2^{800} permutations made it inefficient in finding optimal solutions within a reasonable time frame with this algorithm. Our experiments also

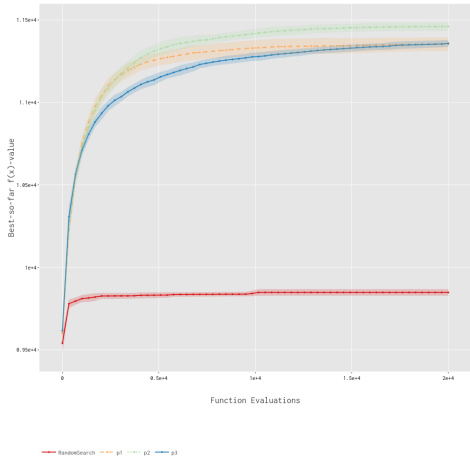
revealed whenever further improvement become marginal. Early on, it was observed that the gains achieved by increasing the number of iterations were substantial. However, whenever the curve "flattened" it became clear that eventually the improvements diminished. This suggests that continuing the optimization process is not worthwhile at this point. Furthermore, the analysis of approaching a particular fitness based on iterations provides valuable insights into the convergence behaviour of the algorithms. Among the different parameter configurations of our implementation of Simulated Annealing, configuration p2 consistently outperformed the others having a higher AUC than the rest and having the best fitness at most points. The accelerated decrease in temperature based on the iteration and previous temperature coupled with the accept function contributed to its superior performance. Finding the correct parameters to improve the performance of our implementation of Simulated Annealing emphasises the difficulty in fine-tuning these randomized algorithms.

Appendix

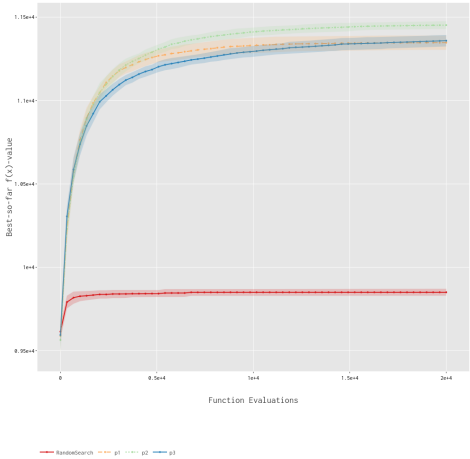
All these figures are shown such that we can draw conclusions from them in our chapter "Results".



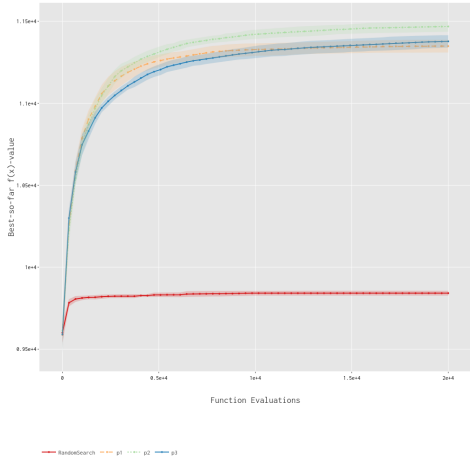
(a) Problem 2000.



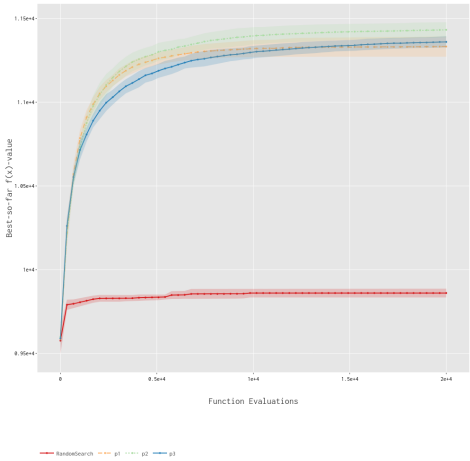
(b) Problem 2001.



(c) Problem 2002.



(d) Problem 2003.



(e) Problem 2004.

Figure 3: The five different problem functions that were evaluated.

References

[1] Iterative optimization heuristics profiler, `iohanalyzer.liacs.nl`, accessed: 14/12/2023

[2] What is simulated annealing?, `https://www.cs.cmu.edu/afs/cs.cmu.edu/project/learn-43/lib/photoz/.g/web/glossary/anneal.html`, accessed: 14/12/2023

[3] Boyd, S.P., Vandenberghe, L.: Convex Optimization. Cambridge University Press (2004)

[4] Bozkaya, B., Erkut, E., Laporte, G.: A tabu search heuristic for political districting. Centre for Research on Transportation= Centre de recherche sur les ... (1999)

[5] D'Amico, S.J., Wang, S.J., Batta, R., Rump, C.M.: A simulated annealing approach to police district design. Computers & operations research **29**(6), 667–684 (2002)

[6] Leiden, U.: Natural computing - universiteit leiden (nd), `https://studiegids.universiteitleiden.nl/courses/116608/natural-computing`, accessed: 14/12/2023

[7] MathWorks: Simulated annealing - matlab documentation (nd), `https://www.mathworks.com/help/gads/what-is-simulated-annealing.html`, accessed: 13/12/2023